SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

SCADA-i is an AI-driven smart building management system (BMS) designed to transform traditional facility monitoring into an autonomous, decision-making platform.

Unlike conventional SCADA systems that only display static IoT data, SCADA-i introduces a supervisory multi-agent AI workflow that interprets real-time sensor and user data, makes intelligent decisions and executes automated actions across building operations.

The system focuses on optimizing energy consumption, improving operational efficiency and reducing manual workload by enabling real-time control of HVAC systems, intelligent room booking coordination and ESG compliance monitoring.

1.1. Purpose:

This System Analysis Document (SAD) outlines the technical architecture, system design and key implementation decisions behind the SCADA-i platform.

The document serves as a reference for developers, stakeholders and evaluators to understand how SCADA-i integrates IoT infrastructure with an agentic AI system to achieve autonomous building management.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *SCADA-i adopts a modular, distributed architecture that integrates IoT devices, backend services and a multi-agent AI orchestration layer powered by Z.AI GLM. The system is deployed across cloud services, with the web interface hosted on Netlify, backend and agent services deployed on Render and data persistence managed via Supabase (PostgreSQL). This separation of components enables scalability, maintainability and real-time system coordination.*
2. ***Data Flows:***
    a. *Real-time sensor data is collected via IoT devices, transmitted through MQTT, processed by backend services and consumed by HVAC agent to generate decisions and trigger HVAC controls.*
    b. *Collected IoT sensor data gets processed by the ESG agent to generate ESG reports, maintenance tickets, and ESG-breach alerts.*

© UMHackathon 2026
Email: umhackathon@um.edu.my

c. *User booking queries and conversational history are routed from the orchestrator to the Booking Agent, which processes the intent and triggers specialized asynchronous tools. These tools execute SQL queries and transactions via asyncpg against a PostgreSQL database to evaluate room and equipment availability, resolve scheduling conflicts, and insert new booking records (including involved departments). Database states are formatted as JSON and consumed back by the Booking Agent to synthesize a final natural language confirmation or schedule summary for the user.*

3. ***Model Process:***
   a. *The system leverages a multi-agent AI workflow where specialized agents (HVAC, Booking, ESG) operate under a central orchestrator to interpret user intent and system conditions, producing structured outputs and executable actions.*
4. ***Role of Reference:***
   a. *This document acts as a technical guide for system development, integration and evaluation, ensuring alignment across all components of the SCADA-i platform.*

1.2. ***Background****:*

Modern buildings generate large volumes of IoT data, including temperature, occupancy and equipment status. However, traditional SCADA systems remain limited to passive monitoring, requiring facility managers to manually interpret data, identify inefficiencies and execute actions.

This results in several operational challenges:

- *Energy waste due to fixed or inefficient control strategies*
- *Delayed response to equipment faults and anomalies*
- *High administrative workload for monitoring, reporting and coordination*
- *Siloed management between building functions such as HVAC control, room booking and ESG compliance*

SCADA-i addresses these challenges by shifting from a data-centric monitoring approach to an action-oriented intelligent system, powered by a multi-agent AI architecture.

The system introduces specialized agents:

- *HVAC Agent to optimize thermal comfort and energy efficiency in real time*
- *Booking Agent to manage room availability, reservations and utilization*
- *ESG Agent to monitor sustainability metrics and enforce policy compliance*

© UMHackathon 2026

These agents are coordinated by a central orchestrator, enabling seamless collaboration across building operations. By continuously analyzing real-time data and user intent, SCADA-i transforms fragmented workflows into a unified system that not only monitors but also decides and acts.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Facility Managers	*Monitor and manage building operations through admin-only dashboard*	*Reduced manual workload, real-time automation and actionable insights*
Office/University Users	*Interact with building services such as booking or comfort control*	*Seamless experience, improved comfort and faster response*
Maintenance Teams	*Handle system faults and repairs*	*Early detection of issues and structured maintenance notification.*
Development Team	*Build the platform and maintain it*	*Clear API usage contracts, modular architecture, clear data flows and scalable design*
QA Team	*Validating the system behavior for all user roles*	*Defined workflows, predictable outputs and reliable system performance on edge cases*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	**Details**

© UMHackathon 2026

Email: umhackathon@um.edu.my

System	*Smart Building Management System (SCADA-i)*
Architecture	*Multi-Agent Microservices with Central Orchestration*

SCADA-i is primarily structured as a unified system in which there are 2 main types of clients. 1. Office/University User Web-based Application, 2. Facility Manager Web-based Dashboard. All these clients communicate by Backend API layer. So, three independent microservices: HVAC Control Service, Room Booking Service and ESG Reporting Service. All agent services are deployed on Render, and all share a PostgreSQL database that is hosted in Supabase (AWS ap-south-1).

2.1.2 LLM as Service Layer

SCADA-i integrates the GLM model (via Z.AI GLM API) as a central reasoning service layer rather than a standalone AI module. The LLM acts as the decision-making interface between user input and system execution.

2.1.3 Dependency Diagram

a. *Prompt Construction and GLM API Transmission*

Prompts are constructed dynamically based on the current state of the conversation and the specific role of the agent:

i. ***Supervisor Prompt:*** *Built using build_supervisor_prompt(), which includes the Team Directory, Delegation Rules and Administrative Gatekeeping (e.g., checking for 'ADMIN' roles).*

ii. ***Worker Prompts:*** *Each worker (Booking, HVAC, ESG) has a specialized system prompt (e.g., HVAC_AGENT_PROMPT) defined in its respective module.*

iii. ***Injection:*** *The Backend API secretly injects metadata, such as the user_id, into the user's message before sending it to the graph: {request.message}\n[SYSTEM NOTE: The current user's ID is '{request.user_id}'].*

iv. ***Transmission:*** *The system uses ChatOpenAI (for the Supervisor/GPT-4o) and ChatDeepSeek (for Workers/DeepSeek-V3) wrappers to send these prompts to the respective LLM APIs.*

b. *Context Window Composition*

The context window sent to the GLM includes:

i. ***System Instructions:*** *The core rules and delegation protocols.*

ii. ***Conversation History:*** *The messages list from the AgentState.*

© UMHackathon 2026

iii. ***Tool Reports:*** *When a worker executes a tool (like checking room availability), the output is appended to the message history as a "report" for the Supervisor to review.*

iv. ***Internal Routing Data:*** *The Supervisor's previous commands to workers.*

c. *Response Parsing and Component Handoff*

The system uses Structured Output to parse responses and determine the next step:

i. ***Parsing:*** *The Supervisor uses with_structured_output(RouterSchema) to force the LLM to return a JSON-like object containing next (the target agent) and command (the specific task).*

ii. ***Routing:*** *A StateGraph conditional edge reads the next field. If it contains "BOOKING_NODE", the graph invokes the booking_node function.*

iii. ***Handoff:*** *The worker processes the command, appends its result to the messages list, and the graph loops back to the Supervisor to decide if the task is complete or if another agent (like HVAC for pre-cooling) is needed.*

d. *Token Enforcements and Input Chunking*

Limitations are enforced at the Orchestrator level before the request reaches the GLM:

i. ***Token Counting:*** *A local_token_counter utility uses the tiktoken library (specifically the gpt-4o encoding) to calculate the length of the message history.*

ii. ***Trimming:*** *The trim_messages function is used in both the Supervisor and Synthesizer nodes. It is configured with max_tokens=4000 and a strategy="last", ensuring only the most recent and relevant parts of the conversation are sent to the API to avoid exceeding context limits.*

e. *Major API Calls and Interactions*

Source	***Component***	***Call/Action***	***Destination***
Frontend	*React App*	*POST /chat*	*FastAPI*
Backend	*FastAPI*	*compiled_app.ainvoke*	*LangGraph*
Orchestrator	*Supervisor*	*structured_llm.oinvoke*	*OpenAI API*
Service 1	*Booking Agent*	*get_room_directory_tool*	*Internal DB*
Service 2	*HVAC Agent*	*MQTT Publish*	*IoT Broker*

© UMHackathon 2026

Persistence	*LangGraph*	*AsyncPostgresSaver*	*PostgresSQL*

f. Frontend ↔ Backend ↔ Services ↔ DB ↔ External API interactions:

i. *Frontend (React/Streamlit) sends user intent to the orchestrator; orchestrator routes to Booking/ESG/HVAC services; Booking persists to PostgreSQL; ESG uses embedding API + Supabase retrieval; HVAC reads/writes state via PostgreSQL and communicates control/optimizer events over MQTT; OpenAI/DeepSeek are invoked for orchestration and reasoning; results return to frontend.*

g. Distributed components:

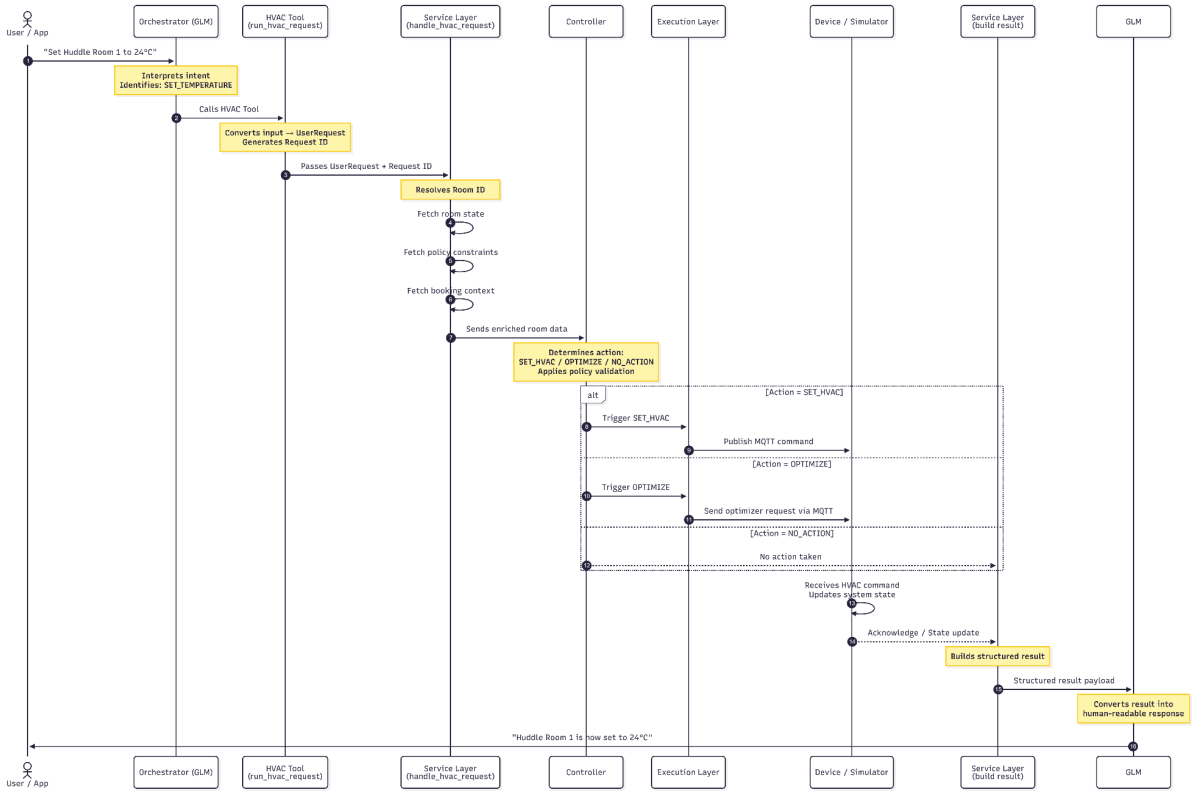
i. *Frontend tier, Orchestrator tier, Worker microservices tier (Booking/ESG/HVAC), Data tier (PostgreSQL + Supabase), Messaging tier (MQTT broker), External AI tier (OpenAI/DeepSeek + embedding API).*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram

Scenario: *HVAC Request*



2.2. Technological Stack

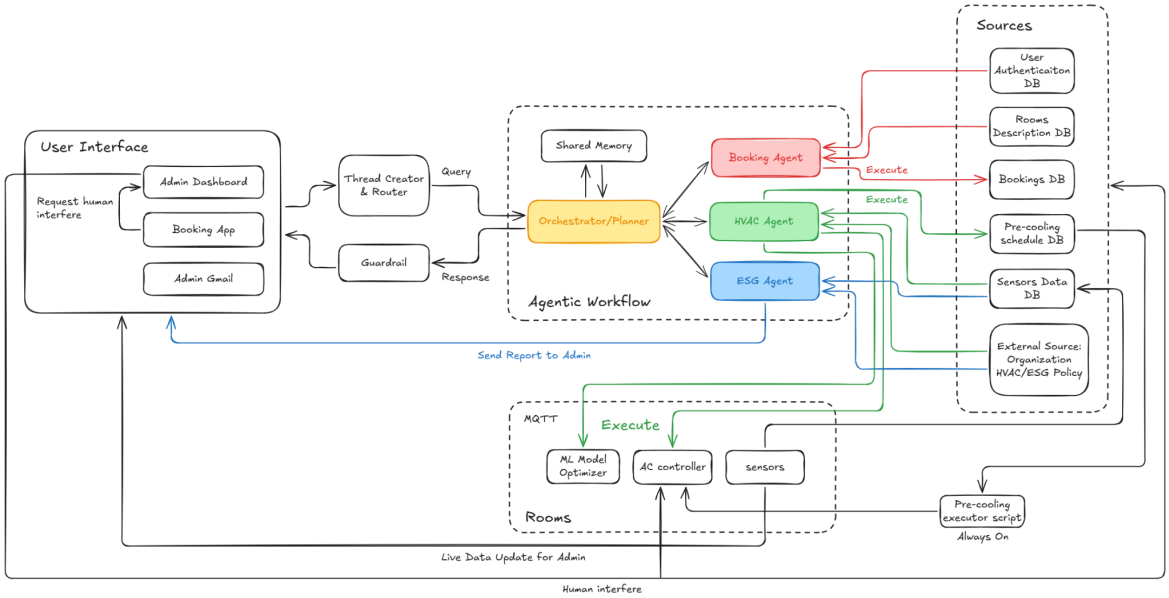
- *2.2.1. Frontend [Framework: React, Language: TypeScript, Build Tool: Vite 7, Styling: Tailwind CSS]*
- *2.2.2. Backend [Runtime/Language: Node.js with TypeScript]*
- *2.2.3. Database [PostgreSQL – Relational Database to store room and hvac states, booking details and ESG policies]*
- *2.2.4. Cloud/Deployment [Netlify (Web Server), Supabase (PostgreSQL), Supabase Storage (Storage Buckets)]*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3. Key Data Flows

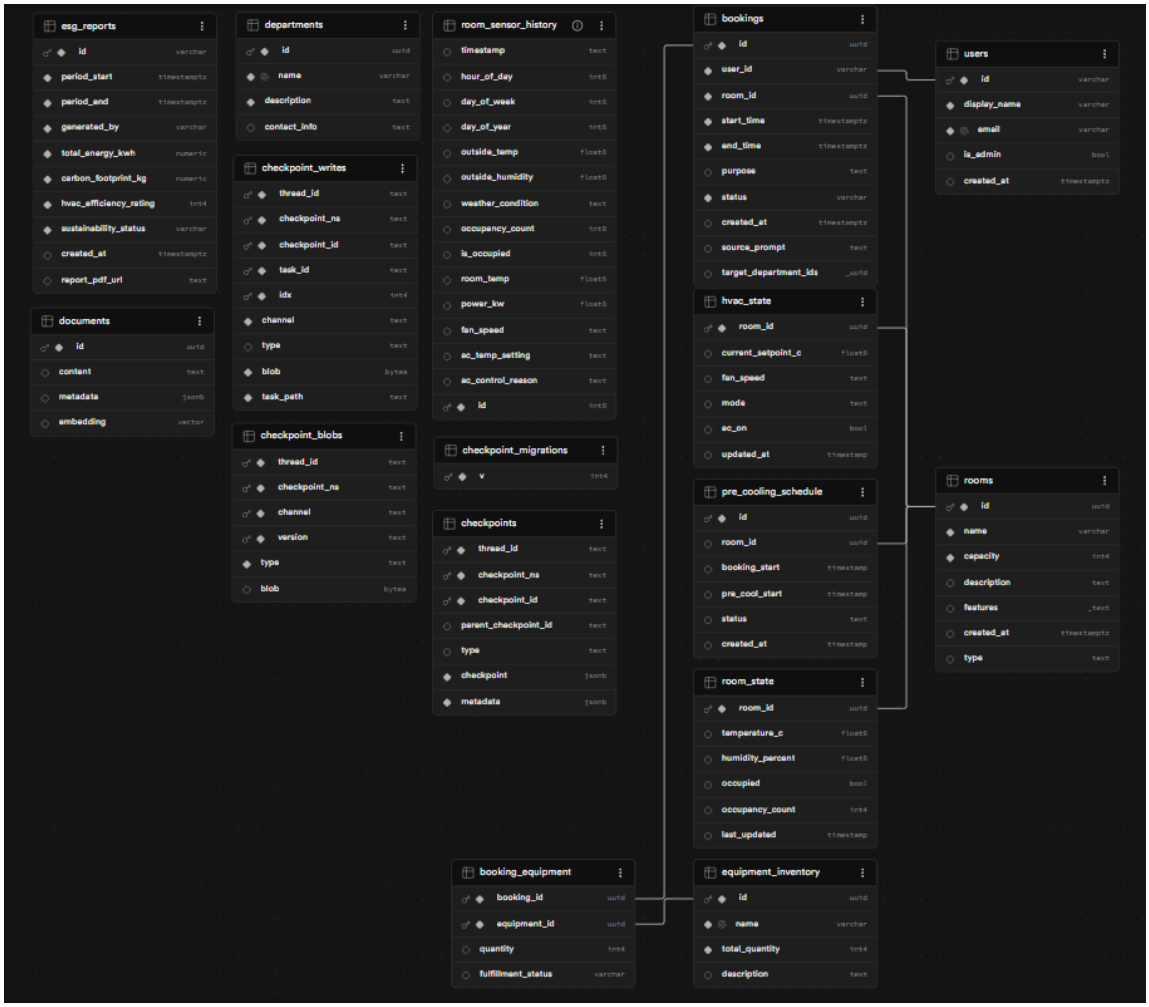
2.3.1.1. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema



3. *Functional Requirements & Scope*

This section defines the core functionalities of SCADA-i and establishes the boundaries of the system. The focus is on delivering a high-impact Minimum Viable Product (MVP) that demonstrates the feasibility of an AI-driven smart building management system.

3.1. *Minimum Viable Product:*

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.*

#	*Features*	*Description*

© UMHackathon 2026

1	*HVAC Intelligent Control*	*The system dynamically adjusts temperature, fan speed and mode based on user requests, occupancy and ESG policies. It supports both manual overrides and automated optimization. The system also automatically prepares rooms before scheduled bookings by triggering HVAC optimization without user input.*
2	*Room Booking Management*	*Users can check room availability, equipment availability and make bookings through natural language interaction. The system ensures proper scheduling and prevents conflicts.*
3	*ESG Monitoring & Compliance*	*The system analyzes environmental conditions and ensures compliance with sustainability policies. Admin can generate reports using the agent, which will return a PDF of said report.*
4	*Anomaly Detection & Alerts*	*Detects abnormal conditions such as sensor failure or inconsistent data and generates alerts to the maintenance team.*

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	***Requirements***	***Implementation***
Scalability	*The system supports multiple rooms and building zones simultaneously.*	*Modular architecture with decoupled services and MQTT-based communication.*
Reliability	*HVAC control and system-triggered actions must execute correctly with minimal failure.*	*Validation in service layer and fallback handling for optimizer failures.*
Maintainability	*Each service like (HVAC, Booking, ESG) is separated and deployed individually.*	*Separation into agents, service layer and infrastructure modules.*

© UMHackathon 2026

Performance Latency	*AI-driven decisions should be returned within a few seconds.*	*Timeout constraints (5–8 seconds) for LLM and optimizer responses.*
Cost Efficiency	*Average costs of token per user session must remain controlled.*	*Structured prompts and minimal context to reduce token usage.*
Real-time Responsiveness	*The system must react to sensor data and user input instantly.*	*Event-driven architecture using MQTT for real-time communication.*

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

a. *Installation of new IoT sensors, hardware or edge nodes across the entire building/campus (testing is strictly limited to the MVP prototype environments).*

b. *Multi-building or campus-wide deployment with centralized monitoring dashboard.*

c. *Voice-to-text processing, speech recognition or audio-based interactions.*

d. *Advanced ESG reporting dashboards with analytics and visualization.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

SCADA-i follows an incremental validation approach to ensure that the system produces reliable and grounded outputs before full deployment:

- *HVAC, Booking and ESG capabilities are validated independently before multi-agent integration.*
- *End-to-end flows are evaluated across success, blocked, fallback and partial-failure paths.*

© UMHackathon 2026

- *Repeated prompt runs are used to verify stable routing behavior, consistent tool invocation, and predictable backend effects.*

4.1.2. Emergency Rollback (ER) & Golden Release:

Optimizer Timeout Handling

If the optimizer does not respond within the defined timeout (5–8 seconds), the system:

- *blocks execution*
- *returns a failure-safe response*
- *avoids applying incorrect HVAC changes*

Manual Override Protection

When a user manually sets a temperature, the system:

- *pauses automated optimization temporarily*
- *prevents conflicting actions*

Fallback Mechanism

If AI or external services fail, the system defaults to:

- *no action (safe state)*
- *clear user-facing explanation*

Booking rollback

- *On booking-write failures, system preserves existing booking state and returns actionable retry guidance.*
- *On conflict detection (slot taken), system does not force commit. It returns alternatives/clarification paths.*

ESG rollback

- *On policy-retrieval failure, system blocks risky compliance-sensitive actions and returns governance-safe responses.*
- *On reporting dependency failure, system returns explicit degraded-mode messaging instead of silent omission.*

4.1.3. Priority Matrix:

4.1.3.1.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Priority	***Condition***	***Action***
P1 (Critical)	*System failure / unsafe HVAC command / policy-bypass attempt*	*Block execution, return safe response, raise operator/admin alert*
P2 (High)	*Optimizer timeout / booking write failure / policy retrieval unavailable*	*Skip dependent step, preserve consistency, return degraded-mode explanation*
P3 (Medium)	*Policy violation (e.g., low temp request), booking conflict, incomplete input*	*Auto-adjust within policy or request clarification/alternative options*
P4 (Low)	*No-op actions, duplicate requests, informational/reporting-only requests*	*Confirm status without unnecessary execution*

*4.2. **Monitoring**:*

4.2.1. SCADA-i includes basic monitoring to ensure system reliability:

- ***System Logs***
  - *All requests are tracked using unique "`request_id`"*
  - *Enables traceability across agent, service, and execution layers*
- ***Execution Tracking***
  - *Records whether actions are:*
    - *Successful*
    - *Blocked*
    - *Failed*
- ***Real-Time State Monitoring***
  - *Room states (temperature, occupancy, etc.) are continuously updated*
  - *Used for decision-making and anomaly detection*
- ***Error Visibility***
  - *Failures (optimizer timeout) are surfaced clearly*
  - *Prevents silent system failures*
- ***Booking monitoring***
  - *Booking lifecycle events logged with booking identifiers and status transitions*
  - *Conflict and validation failures are observable for operational tuning*

© UMHackathon 2026

Email: umhackathon@um.edu.my

    - *Audit trail for update/cancel operations improves recoverability and compliance*
  - ***ESG monitoring***
    - *Policy lookup attempts and retrieval outcomes logged for explainability*
    - *Compliance decision outcomes logged for audit and governance checks*
    - *Reporting access-denied events logged for role-policy enforcement analysis*

4.3. Assumptions:

The following assumptions were made during system design and implementation:

- *IoT sensors provide accurate and continuous data*
- *MQTT communication between system components is stable*
- *Agents servers are stable*
- *System operates in a controlled environment*
- *Calendar/room inventory data is reasonably up-to-date*
- *Room capacity and attributes are maintained with acceptable data quality*
- *User identity context is available for ownership/permission checks*
- *Policy documents are correctly ingested and indexed for retrieval*
- *Governance thresholds/rules are current and approved by policy owners*
- *Role mappings (admin vs non-admin) are synchronized with access controls*

4.4. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Core LLM reasoning and decision-making layer that will handle prompt inference, multi-step reasoning and generating structured output	High – if the API fails or is rate-limited, the agent cannot process requests. However, there will be graceful degradation state from users

© UMHackathon 2026
Email: umhackathon@um.edu.my

Policy retrieval stack (embedding + vector store)	*Supports ESG grounding*	*High - retrieval miss/irrelevant chunks can degrade compliance confidence*
Role/identity provider or user registry	*Supports reporting authorization controls*	*High - desynchronized roles may cause incorrect access decisions*
MQTT Broker	*Real-time communication between system components*	*Medium – message delays or failures may affect responsiveness*
Supabase / PostgreSQL	*Data storage for room state, bookings, ESG policies and logs*	*Medium – database downtime affects system state consistency*
Optimizer Module (LightGBM)	*Generates HVAC optimization recommendations*	*Medium – failure leads to fallback (no optimization)*
Booking data services / calendar backends	*Supports availability checks and mutation operations*	*Medium - stale/inconsistent data can produce false availability or failed writes*
Device Simulator	*Simulates HVAC system execution*	*Low – only affects simulation, not core logic*

5. Project Management & Team Contributions

5.1. Project Timeline:

Phase	**Activities**
Day 1–2: Planning & Design	*Problem analysis, solution ideation, system architecture design and task allocation*
Day 3–6: Core Agents, Frontend and Backend Development	*Implementation of backend services, data models, IoT simulation setup and development of HVAC, Booking,*

© UMHackathon 2026

	ESG agents and mobile + dashboard development
Day 7–8: AI Agent Integration	*Integrations of all sub-agents with Z.AI GLM API as the core reasoning engine*
Day 9–10: Testing & Refinement	*End-to-end testing, debugging, scenario validation and presentation with demo preparation*

5.2. *Team Members Role:*

Member	*Role*	*Responsibility*
Ho Yu Xiang	*Agentic AI Developer Lead*	*Led the architecture design and continuous refinement for the Agentic AI system. Developed booking and orchestration logic to enable coordination across agents. Oversaw prompt engineering, tool integration, and decision pipelines to ensure alignment with system objectives.*
Sang Yen Ling	*Frontend Developer*	*Designed initial wireframe to kickstart the project. Coded the frontend and worked closely with Backend Developer to ensure synchronization.*
Muhammad Safrul bin Roslan	*Project Lead & Sub Agent Developer*	*Coordinated whole team to ensure daily KPIs are met. Designed ESG agent (tools, decision logic). Oversaw the whole project and approve final deliverables.*
Fadzlee Adam bin Mohd Nazlee	*Backend Developer*	*Designed the backend infrastructure to support core application functionality (including API development and system integration).*

© UMHackathon 2026

		Managed deployment and hosting of applications on cloud platforms.
Teoh Jay Rong	*Sub Agent Developer & Documenting*	*Designed and implemented the HVAC agent including its own tools, decision and coordination logic, schemas, pre-cool scheduling logic and alert mechanism. Provided initial drafting for pitch deck, Product Requirement Document (PRD) and System Analysis Document (SAD).*

5.3. ***Recommendations:*** *To enhance scalability, performance and system robustness, the following improvements are recommended:*

- ***Scalability Enhancement***
  Extend the system to support multi-building deployment with centralized monitoring
- ***Performance Optimization***
  Introduce caching mechanisms (Redis) to reduce repeated API calls and latency
- ***Reliability Improvement***
  Implement fallback strategies and redundancy for critical components such as the LLM API and MQTT broker
- ***Advanced Analytics***
  Integrate predictive models for long-term energy optimization and maintenance forecasting

© UMHackathon 2026